

Referral Robin — Replit Agent

Build Brief

App overview

Referral Robin is a fair-rotation referral code marketplace. Users submit referral codes for brands (Monzo, Revolut, Starling, etc.). Visitors pick a brand, get served the next code in a weighted round-robin queue, copy it to clipboard, and face a 10-minute per-device cooldown before they can pull another code from that same brand. Codes are never shown in a public list — always served one at a time, server-side, so they can't be scraped or mass-copied.

Tech stack

- **Frontend:** Flutter (Android first, iOS later) — matches existing Time Value stack
- **Backend:** Node.js (Cloud Functions or standalone Express) — codes must never be queried directly from the client

- **Database:** Firestore
- **Auth:** Firebase Auth (email + Google sign-in minimum)
- **Payments:** Google Play Billing (premium subscription / code boosts) – same pattern as Time Value, not Stripe
- **Hosting:** gettimevalue-style setup – Replit for backend/dev, Firebase Hosting for any web landing page at referralrobin.com

Data model

```
brands/{brandId}
  name, logoUrl, currentOffer,
  offerUpdatedAt, active, category

codes/{codeId}                // NEVER
exposed directly to client
  brandId, code, ownerId, status, weight,
  tier,
  timesServed, timesCopied, lastServedAt,
  reportCount

queue_state/{brandId}
  order: [codeId...]          // weighted –
premium codes appear multiple times
  cursor

device_cooldowns/{deviceId}_{brandId}
  lastCopyAt, expiresAt
```

```
users/{userId}
  email, authProvider, createdAt, tier,
  premiumExpiresAt

sponsored_slots/{category}
  advertiserName, imageUrl, linkUrl,
  expiresAt
```

Core Cloud Functions

1. getNextCode(brandId, deviceId)

- Check `device_cooldowns` — reject if not expired
- Pull next codeId from `queue_state.order[cursor]`, advance cursor
- Update `codes.lastServedAt`, `timesServed`
- Write new `device_cooldowns` doc, `expiresAt` = now + 10min
- Return code + brand's current offer amount only

2. confirmCopy(codeId, deviceId) — fired when user taps "copy," increments `timesCopied` (distinct from `timesServed`, so you can measure real usage vs. just views)

3. **submitCode(userId, brandId, code)** — adds to `codes`, triggers a rebuild of that brand's `queue_state.order` to insert it at correct weighted spacing
4. **updateCodeWeight / setPremiumTier** — triggered on subscription purchase webhook, rebuilds queue order for affected codes
5. **rebuildQueue(brandId)** — helper, regenerates the weighted order array (weight 1 = 1 slot, weight 3 = 3 slots interleaved, not stacked back-to-back)

Screens (MVP)

1. **Brand picker** — grid of brand logos, search/filter by category
2. **Code reveal screen** — big offer amount, "Copy Code" button (grays out during cooldown, shows countdown), brand's T&Cs link
3. **Sign up / login** — email + Google
4. **My codes dashboard** — list of user's submitted codes per brand, edit button per code, stats (times served / times copied) per code and lifetime total
5. **Submit a code** — pick brand, paste code, submit

6. **Premium upsell screen** — explain weighting benefit, Play Billing purchase flow
7. **Report a dead code** — simple flag button on reveal screen

Build phases

Phase 1 — Core loop (MVP)

- Brand list, code submission, weighted queue (weight=1 for everyone initially), cooldown, copy-to-clipboard, basic auth, dashboard with stats

Phase 2 — Premium

- Play Billing integration, weight boost on purchase, premium badge in dashboard

Phase 3 — Ads

- Sponsored slot component above brand grid, simple admin panel (manual entry is fine at first — no need for a self-serve ad platform yet)

Phase 4 — Trust & safety

- Dead code reporting → auto-deprioritise after N reports, admin review queue

Notes for Replit Agent prompt

When you drop this in, be explicit that:

- Codes must route through a Cloud Function, never a direct Firestore client read
- Device ID should be a persisted local UUID (SharedPreferences), not IMEI/ad-id
- Cooldown is per-device-per-brand, not per-code
- Queue weighting must interleave (not block) premium codes into the rotation